BasTo6809 Compiler

User Manual and Language Reference

Compiler version 5.45 · Glen Hewlett · August 2026

Contents

About this manual

This edition reorganizes the original BasTo6809 documentation into a maintainable reference guide. Command syntax, examples, memory-layout tables, hardware notes, and example programs are retained from the original manual while typography, navigation, terminology, and page structure are standardized.

> **Note:** Compatibility note: Some commands require a specific Color Computer model, CoCoSDC, CoCoMP3, or compiler option. Check each command entry and the applicable hardware section before use.

Document conventions

- Command signatures appear in a shaded monospaced heading.
- Examples appear in monospaced code blocks and use compiler-ready straight quotation marks.
- Hexadecimal values use the compiler's &H prefix or the 6809 assembler's $ prefix, depending on context.
- A # placeholder denotes a numeric argument unless the command entry states otherwise.

Quick Start

1. Unzip the BASIC_To_6809 package on your computer.
2. Open a terminal and change to the new BASIC_To_6809 folder.
3. On macOS, you may need to remove the downloaded-file quarantine before the included executables will run. From the BASIC_To_6809 folder, enter: xattr -d com.apple.quarantine * 2>/dev/null. Windows users can skip this step.
4. Start the Super Duper Extended Color BASIC IDE with ./SDECB. On Windows, use SDECB.exe.
5. Enter a BASIC program in the IDE or load TEST.BAS from the File menu. The IDE checks syntax and helps find typing errors before compilation.
6. Choose a compile command from the Compile menu, or press F11 for the default build configured in the makefile (compile.bat on Windows).
7. The default build compiles the BASIC source to 6809 assembly, assembles it with lwasm, creates DISK1.DSK from BLANK.DSK, and copies the generated program to the disk image with decb.
8. Run the disk image in an emulator or copy it to an SD card for use on a real CoCo with a CoCoSDC. The makefile and compile.bat are intended to be customized for your workflow.

Once TEST.BIN builds successfully, you are ready to compile your own BASIC programs. Have fun!

Command Line Options

BasTo6809 accepts the options below. Options placed in a CompileOptions comment at the start of a .BAS file take precedence over command-line options. Example:

EXAMPLE

```
'CompileOptions –i –notext –r80 –s32 –o2 –fArcade_B0_F1
```

–coco

Read a tokenized Color Computer BASIC input file.

–ascii

Read a plain-text ASCII BASIC input file. This is the normal choice for files created in a text editor or the SDECB IDE.

–dragon

Generate output for a Dragon computer.

–mx

Select floating-point precision: -m0 uses the faster 3-byte format; -m1 uses the more accurate 5-byte format.

–i

Use strict integer-only compilation. Default numeric variables are INTEGER, integer division truncates toward zero, and floating-point literals, types, operators, or functions produce compile-time errors. Floating-point libraries are omitted. LONG, _INTEGER64, and unsigned integer types remain available and include only the integer support they require.

```
-sxxx
```

Set the maximum size of ordinary strings, from 1 through 255 bytes. The default is 255. Fixed-length declarations such as DIM NAME$ AS STRING * 32 can set a smaller maximum for an individual string; assignments are truncated to the declared maximum.

```
-o0 / -o1 / -o2
```

Select the optimization level. The default is -o2; -o0 disables optimization.

```
-notext
```

Do not reserve the standard $0400-$05FF text screen. The low-memory layout begins at $0400 instead of $0600. This option cannot be combined with the CoCo 3 WIDTH command.

```
-pxxxx
```

Set the compiled program's start address to hexadecimal xxxx. The address must be at or above the safe address calculated from resident loaders, shared RAM, and graphics pages. The compiler reports an error instead of silently moving an unsafe address.

```
-fxxxx
```

Select the graphics-screen font. The default is Arcade_B0_F1. Available fonts are in Basic_Includes/GraphicsMode/Graphic_Screen_Fonts.

```
-a
```

Make the generated program autostart after loading.

```
-r
```

Return to BASIC when the compiled program ends instead of entering the normal endless loop.

> -Vx

Set compiler verbosity. -V0 produces minimal output; higher values provide progressively more detail.

> -v

Display the BasTo6809 version number.

> -k

Keep miscellaneous intermediate files generated during compilation.

> -h

Display command-line help.

> -rxxxx

Reserve xxxx hexadecimal bytes of shared RAM between resident loaders and graphics memory. The area is left untouched so CHAINed programs can share variables, pointers, tables, or other data. Chained programs must use a compatible layout.

Resident loaders

-rxxxx shared RAM

Alignment to the next $0200 boundary

Graphics memory

Alignment to the next $0100 boundary

Compiled program

```
All values supplied to -rxxxx are hexadecimal.
```

No reserved RAM

Options and commands	Loader locations	Graphics start	Program start without graphics
Default, no CHAIN	—	$600	$600
-notext, no CHAIN	—	$400	$400
SDC_CHAIN	SDC: $0600–$07A5	$800	$800
-notext, SDC_CHAIN	SDC: $0400–$05A5	$600	$600
Disk CHAIN	Disk: $0600–$09EF	$0A00	$0A00
-notext, disk CHAIN	Disk: $0400–$07EF	$800	$800
Both loaders	SDC: $0600–$07A5; Disk: $07A6–$0B95	$0C00	$0C00
-notext, both loaders	SDC: $0400–$05A5; Disk: $05A6–$0995	$0A00	$0A00

With -r80

```
-r80 reserves $0080, or 128 bytes.
```

Options and commands	Loaders end	Reserved shared RAM	Graphics start	Program start without graphics
-r80, no CHAIN	$600	$0600–$067F	$800	$800
-notext -r80, no CHAIN	$400	$0400–$047F	$600	$600
-r80, SDC_CHAIN	$07A6	$07A6–$0825	$0A00	$0A00
-notext -r80, SDC_CHAIN	$05A6	$05A6–$0625	$800	$800
-r80, disk CHAIN	$09F0	$09F0–$0A6F	$0C00	$0C00
-notext -r80, disk CHAIN	$07F0	$07F0–$086F	$0A00	$0A00
-r80, both loaders	$0B96	$0B96–$0C15	$0E00	$0E00
-notext -r80, both loaders	$996	$0996–$0A15	$0C00	$0C00

GMODE 16 examples

```
One GMODE 16 page consumes $1800 bytes.
```

Configuration	Reserved shared RAM	Graphics memory	Automatic program start
Default, GMODE 16	—	$0600–$1DFF	$1E00
-notext, GMODE 16	—	$0400–$1BFF	$1C00
-notext -r40, GMODE 16	$0400–$043F	$0600–$1DFF	$1E00
-notext -r80, GMODE 16	$0400–$047F	$0600–$1DFF	$1E00
-notext, both loaders, GMODE 16	—	$0A00–$21FF	$2200
-notext -r80, both loaders, GMODE 16	$0996–$0A15	$0C00–$23FF	$2400
-r80, both loaders, GMODE 16	$0B96–$0C15	$0E00–$25FF	$2600

```
The reason –notext –r40 moves the graphics start from $0400 to $0600 is:
$0400–$043F   Reserved shared RAM
$0440–$05FF   Alignment space
$0600         Next valid $0200-aligned graphics address
An explicit –pxxxx may place the program at a higher address, but it cannot overlap the
loaders, shared area, or graphics pages.
If using CoCo 3 WIDTH command, you cannot use the –notext option:
WIDTH 80 uses $1220 bytes:
80 columns × 29 rows × 2 bytes = 4640 bytes = $1220
The layout order is:
CHAIN loaders
```

-rxxxx shared RAM

Align to $0200

WIDTH 80 screen

Align to $0100

Compiled program

WIDTH 80 without -rxxxx

BASIC commands used	Loader memory	WIDTH 80 screen	Program start
No CHAIN loader	—	$0600–$181F	$1900
SDC_CHAIN	$0600–$07A5	$0800–$1A1F	$1B00
Disk CHAIN	$0600–$09EF	$0A00–$1C1F	$1D00
Both loaders	$0600–$0B95	$0C00–$1E1F	$1F00

WIDTH 80 with -r80

```
-r80 reserves $80, or 128 bytes.
```

BASIC commands used	Loader memory	Shared -r80 RAM	WIDTH 80 screen	Program start
No CHAIN loader	—	$0600–$067F	$0800–$1A1F	$1B00
SDC_CHAIN	$0600–$07A5	$07A6–$0825	$0A00–$1C1F	$1D00
Disk CHAIN	$0600–$09EF	$09F0–$0A6F	$0C00–$1E1F	$1F00
Both loaders	$0600–$0B95	$0B96–$0C15	$0E00–$201F	$2100

```
For any -rxxxx value:
Shared start = first byte after the loaders
Shared end   = shared start + $xxxx - 1
Screen start = next $0200 boundary after shared memory
Program start = next $0100 boundary after screen start + $1220
For example, with both loaders and -r200:
```

Region	Address
SDC loader	$0600–$07A5
Disk loader	$07A6–$0B95
Shared RAM	$0B96–$0D95
Alignment	$0D96–$0DFF
WIDTH 80 screen	$0E00–$201F
Program	$2100 onward

```
–notext cannot be combined with WIDTH 80 or other WIDTH commands; that combination produces a compile-time error.
```

Enhancements to Colour BASIC

– Write BASIC programs using the helpful included Super Duper Extended Colour Basic (SDECB) IDE.

– Using the OPTION _EXPLICIT feature in the SDECB IDE it will show you unassigned variables, making it easy to catch typos.

– New GMODE command allows you to choose every graphic mode the CoCo can produce, including semi graphics and if using a CoCo 3 all of the CoCo 3 graphics modes and Colour modes. Using these new screens you can use LINE (with B & BF), CIRCLE and PAINT commands.

- Can print directly to any graphic screen using PRINT #-3,
- Use of line numbers is optional
- You can use Labels for sections of code to jump to (case sensitive)
- Variable names can be 25 characters long (case sensitive)

– Doesn't use any ROM calls (Extended BASIC ROM not required), possible to use all of the 64k of RAM on Coco 1 & 2

- Added new sprite commands

– Added commands for the CoCo 3 to use scrollable playfields (backgrounds)

- Added new audio sample commands

– Many new SDC related commands allow you to read and write directly to the SDC filesystem from your BASIC program

– A new SDC audio streaming command to play RAW music files/audio samples directly from the SD card in the CoCoSDC

– Easily insert your own assembly code anywhere you want in your program and easily share values of variables between BASIC and your assembly code.

– Added keyword Const which let's you label a value and use the value in your program, but doesn't take any RAM in your final compiled program.

– The assembly language code generated is fully commented showing each BASIC line and how it's compiled. The assembly file generated can be used to help someone learn how to program in assembly language. Or allow an experienced assembly programmer to optimize the program by hand.

Changes to BASIC's Graphic features

PMODE has been replaced by the GMODE command PCLS has been replaced by the GCLS command PCOPY has been replace with the GCOPY command LINE command format has been changed to include a colour value and no longer uses PSET and PRESET.

The commands PSET, HSET, PRESET, HRESET, PPOINT or HPOINT are not supported. Instead they are replaced with SET and POINT commands. The compiler will use whichever graphic mode is set using the GMODE command and will SET pixels to the requested colour the user wants that matches the GMODE requested.

You can now use SET,POINT,LINE,CIRCLE & PAINT commands on every screen, even the regular text screen, using GMODE 0,1

GMODE ModeNumber,GraphicsPage Selects the graphics screen and the graphics page. ModeNumber is the graphics mode you want to use GraphicsPage is the Page you want to show/use for your graphics commands To see a list of ModeNumbers and the resolutions go here Special note the ModeNumber must be an actual number and cannot be a variable as the compiler needs to know exactly which graphic mode commands to be included at compile time. GraphicsPage can be a variable. If you are going to use Graphic pages, the compiler needs to know how many pages to reserve in RAM (for CoCo 1 & 2 graphics). So you must have a GMODE #,MaxPages entry at the beginning of your BASIC program. Where the value of MaxPages will be an actual number and not a variable.

```
GCLS #
```

Colour the graphics screen # is the colour value you want the screen to be coloured

GCOPY SourcePage,DestinationPage

Makes a copy the Source graphics page to the Destination graphics page. SourcePage - Source graphics page DestinationPage - Destination graphics page

```
SET(x,y,Colour)
```

Sets a pixel on the screen x,y - Screen location of the pixel to be drawn

Colour - Colour Number of the pixel to be drawn

POINT(x,y) Returns the colour value of the pixel selected x,y - Screen location of the pixel value requested

```
LINE(x0,y0)-(x1,y1),Colour[,B][F]
```

x0,y0 - Starting location x1,y1 - Ending location Colour - Colour of the Line or Box to draw B - Draw a Box F - Fill the Box

```
PAINT(x,y),OldColour,FillColour
```

Fills the old colour value with the fill colour value which must also be the border colour of the section you are painting x,y - Starting location OldColour - Colour Number FillColour - Colour Number

```
CIRCLE(x,y),Radius,Colour
```

Draws a circle on the screen x,y - Origin of the circle Radius - Size of the circle, to keep the aspect ratio close to round Some of the graphics modes use scaling so the Radius isn't always a count of actual pixel values. Colour - Colour Number of the circle to draw

```
PALETTE v,Colour
```

Sets the CoCo 3 Palette value

v -palette slot of 0 to 15

Colour - Colour value of 0 to 63

GET & PUT commands are not available

New commands and features added to Color BASIC

- IF/THEN/ELSE/ELSEIF/ENDIF
- SELECT/CASE
- WHILE/WEND
- DO/WHILE/LOOP
- DO/LOOP/UNTIL
- COCOHARDWARE a Command that let's you identify which CoCo your program is running on V=COCOHARDWARE(0)
- COCOMP3 Commands See here for more info

– SDC_PLAY Command that plays an audio sample or song directly off the SD card in the SDC Controller. See here for more info

– SDC_PLAYORCL, SDC_PLAYORCR, SDC_PLAYORCS these commands are similar to SDCPLAY except the audio is sent to the Orchestra 90 or COCOFLASH cartridge. See here for more info

– SDC file access commands that allow you to Read & Write files directly on the SD card's own filesystem. See here for more info

– GETJOYD - Quickly get the joystick values of 0,31,63 of both joysticks both horizontally and vertically

- PRINT #-3, - Print to the graphics screen created with the GMODE command. Can also be used as ?#-3,"Hello World!". The default font is ArcadeArcade_B0_F1, but you can select others using the compiler command -fxxxx Where xxxx is the font name used for printing to the graphics screen (default is Arcade_B0_F1). Look in folder Basic_Includes/GraphicsMode/Graphic_Screen_Fonts to see font names available
- SLEEP Used to pause program execution
- PLAYFIELD # - Used to set the Scrollable playfield mode. The # given must be an actual number and not a variable. # Max Resolution Min Resolution Size Multiple 1 - 256 x 7872 256 x 192 not applicable x 64 2 - 512 x 3840 512 x 192 not applicable x 192 3 - 1024 x 512 1024 x 256 not applicable x 256 4 - 2048 x 256 512 x 256 256 x not applicable 5 - 2560 x 192 512 x 192 256 x not applicable
- VIEW x,y - Command to select where in the scrollable playfield you want to see. The x & y co-ordinates are the top left corner of a viewable window of the playfield.

SPRITE_LOAD "SpriteName.asm", #[,f] — Load a Sprite The SPRITE_LOAD command loads a sprite into your program. • "SpriteName.asm" is the name of your sprite file (must be in assembly format). Generated by the command line tool called PNGtoCCSprite • # is the sprite number — a numeric ID used to reference this sprite in your program when using actual SPRITE commands. • f is the number of frames this sprite has can be left blank if the sprite has only one frame. This command associates the loaded sprite with the given number, so you can easily manage multiple sprites in your game.

SPRITE Command — Control Sprites on the Screen The SPRITE command, along with various options, allows you to control the appearance and behaviour of sprites in your program.

Command Reference:

```
SPRITE OFF [#]
```

Turns off sprite number #.

```
If no number is provided, all sprites are turned off.
SPRITE LOCATE #, x, y
```

Moves sprite # to a new position on the screen. x and y are the playfield coordinates.

```
SPRITE SHOW #[, f]
```

Displays sprite # using frame f.

```
If the sprite is not animated (i.e. it has only one frame) use of ,f is optional.
SPRITE BACKUP #
```

Saves the background behind sprite #. This allows the sprite to be cleanly erased later.

```
SPRITE ERASE #
```

Erases sprite # and restores the background previously saved with SPRITE BACK.

```
SPRITE #,x,y[,f]
```

Draws sprite number # at x,y co-ordinates using frame f. f is optional, a value of 0 (single frame sprite) is used if not given.

WAIT VBL - Wait for Vertical Blank then update the sprites on screen

TRIM$(string) Removes spaces from both ends of a string string - String variable

LTRIM$(string) Removes spaces from the beginning of a string string - String variable

RTRIM$(string) Removes spaces from the end of a string string - String variable

Adding Assembly code to your program

You can insert your own assembly code directly in your BASIC program anywhere you like. To insert your own assembly code into the program you can do so by using the special words "ADDASSEM:" and "ENDASSEM:" after a REM (‘) symbol and a space as:

‘ ADDASSEM:

```
LDA    #$FF
```

RORB ‘ ENDASSEM:

Another example of putting your own assembly code in the BASIC program:

```
Print "HEY"
GoSub Mycode
Print "HELLO"
...
Mycode:
' ADDASSEM:
LDA    #$FF
STA    $400
' ENDASSEM:
Return
```

Accessing Numeric variables in your assembly code

You access numeric variables by adding the prefix "*Var*" to the name. For example if you have an integer variable in your program called Counter and you want to read the value of the variable you could use code like this:

```
DIM Counter As Integer
PRINT "COUNTER=";Counter
' ADDASSEM:
LDD _Var_Counter    ; Get the variable in D
ANDB    #%11111110      ; Make the number even
STD _Var_Counter    ; Save the updated value
' ENDASSEM:
PRINT "COUNTER=";Counter
```

Accessing String variables in your assembly code

You access string variables by adding the prefix "*StrVar*" to the name. Strings are stored as data where the first byte is the length of the string the rest of the bytes are the actual string data.

```
For example if you want to interact with a string variable called A$ in assembly you
could use code like this:
' ADDASSEM:
LDX #_StrVar_A  ; X points at the first byte in the A$ variable
LDB   ,X+           ; B = value of the first byte of the string, X=X+1
; First byte is the length of the string 0 to 255
BEQ   @Finished     ; If the string length is zero then skip past
LDU   #$400         ; Text screen start
!     LDA ,X+        ; Get a byte from the string
STA   ,U+           ; Show the byte on screen
DECB                ; Decrement the counter
BNE   <             ; Keep looping until B=0
@Finished:              ; @ in the label makes it a local label
; always leave a blank line after a section of code
; that uses the @ sign as a local label
' ENDASSEM:
```

Accessing Arrays in your assembly code

You access string variables by adding the prefix "*StrVar*" to the name. Strings are stored as data where the first byte is the length of the string the rest of the bytes are the actual string data.

```
For example if you want to interact with a string variable called A$ in assembly you
could use code like this:
Dim Buffer(1024) As _Unsigned _Byte
Dim BufferStart As _Unsigned Integer
10 GoSub DoStuff
DoStuff:
' ADDASSEM:
LDX #_ArrayNum_Buffer
STX _Var_BufferStart
' ENDASSEM:
Return
```

Compiling from the command line

It is highly recommended that you use the SDECB IDE as it will show you syntax errors and other errors that will help to compile your program without errors. You can also compile your BASIC programs to machine language without the IDE, from the command line using the following:

```
Using MacOS or Linux:
./BasTo6809 -ascii BASIC.bas
lwasm -9bl -p cd -o./ML.bin BASIC.asm > ./Assembly_Listing.txt
Using Windows:
.\BasTo6809.exe -ascii BASIC.bas
lwasm -9bl -p cd -o./ML.bin BASIC.asm > ./Assembly_Listing.txt
```

At this point you'll have an EXECutable program called ML.bin in the folder that you can use on a real CoCo or an emulator.

Optimizing

The 6809 CPU has the hardware MUL instruction and it is used for many of the multiplication routines even the large 16, 32, 64 bit integer and the floating point math routines use it. Division is always going to be slower as it is all software based. Therefore if you want to maximize the speed of your program, try to use multiplication instead of division if possible.

```
For example if you have something like this:
```

A = B / 2

```
Use:
```

A = B * .5

If variables are not assigned a type using the DIM command they will default to Single (Fast Floating Point). To maximize the speed of the 6809 CPU try to use 8 bit (fastest) or 16 bit types (fast). Other variable types will take more space and will run slower. See the next page for all the variable types.

Variable Types

Normally, a numeric variable whose type cannot be inferred defaults to SINGLE. With -i, it defaults to INTEGER and floating-point use is rejected at compile time. It's best if you assign variables with the DIM statement or use the symbol values as a suffix after your variable name or literal number, to use the minimum size needed.

Type	Suffix	Declaration	Range
1	`	_Bit	Min -1, Max 0
2	~`	_Unsigned _Bit	Min 0 , Max 1
3	%%	_Byte	Min -128, Max 127
4	~%%	_Unsigned _Byte	Min 0, Max 255
5	%	Integer	Min -32,768, Max 32,767
6	~%	_Unsigned Integer	Min 0, Max 65,535
7	&	Long	Min -2 Gig, +2 Gig
8	~&	_Unsigned Long	Min 0, Max 4 Gig
9	&&	_Integer64	Min -(8 byte value), Max +(8 byte value)
10	~&&	_Unsigned _Integer64	Min 0, Max (8 byte value)
11	!(None)	Single (Default size)	Min E-20, Max E+19 (Fast Float)
12	#	Double	Min E-308, Max E+308

*** Types 7 to 12 will slow your program down considerably.

Examples of assigning variable types:

```
DIM MyVar1 AS _Byte
DIM MyVar2 AS _UNSIGNED LONG
DIM MyVar3 AS DOUBLE
```

Generally you don't need to force a variable to a different type then assigned, the compiler will scale the types between each other. But this is an example of forcing the type:

A = V~%% ‘ Force V to be an Unsigned Byte

64k programs

Use `cc1sl` (CoCo 1 Super Loader) when the first CoCo program requires more than 32K. Also use it when Disk BASIC starts a compiler-generated program containing `CHAIN` or `SDC_CHAIN`, even if the main program is small: the resident chain loader occupies low RAM used by Disk BASIC while loading. `cc1sl` creates a startup file that safely handles both cases, including programs that use addresses normally occupied by BASIC ROM.

Dragon programs compiled with `-dragon` do not require this step. `DRAGLOAD.BIN` handles 64K Dragon programs directly.

cc1sl usage

```
cc1sl [-l] [-vx] FILENAME.BIN -oOUTNAME.BIN [.scn | .csv]
```

- `-l` displays `LOADING` at the bottom of the screen while the program loads.
- `-vx` selects verbosity 0, 1, or 2. The default is `-v0`.
- `FILENAME.BIN` is the assembled CoCo machine-language program.
- `-oOUTNAME.BIN` sets the output filename; the default is `GO.BIN`.
- An optional `.scn` file supplies a binary CoCo text screen displayed while loading.
- An optional `.csv` file supplies a CSV text screen displayed while loading.

See `cc1sl_help.txt` for the complete utility reference.

The steps for compiling a big program are:

```
For MacOS and Linux:
./BasTo6809 HELLO.BAS
lwasm -9bl -p cd -o./HELLO.BIN HELLO.asm > ./NEW_Assembly_Listing.txt
./cc1sl -l HELLO.BIN -oBIGFILE.BIN
For Windows:
.\BasTo6809 HELLO.BAS
lwasm -9bl -p cd -o./HELLO.BIN HELLO.asm > ./NEW_Assembly_Listing.txt
.\cc1sl -l HELLO.BIN -oBIGFILE.BIN
```

In this case your final program to execute on the CoCo is called BIGFILE.BIN, you can of course call it whatever you want.

Preparing programs for CHAIN and SDC_CHAIN

`CHAIN` and `SDC_CHAIN` replace the running program with another compiled DECB-format machine-language program. Both resident chain loaders can load large, multi-segment programs throughout the 64K address space. This changes where `cc1sl` belongs in a chain-loading project:

Program in the sequence	File to use	Run through cc1sl?
First compiler-generated program containing `CHAIN` or `SDC_CHAIN`, initially loaded from Disk BASIC	The `cc1sl` output	Yes
First program without a resident chain loader and entirely within the normal Disk BASIC loadable area	Normal assembled `.BIN`	No
Program loaded by `CHAIN`	Normal assembled `.BIN`	No
Program loaded by `SDC_CHAIN`	Normal assembled `.BIN`	No

Only the initially launched program may use the `cc1sl` wrapper. When Disk BASIC launches a compiler-generated program containing a resident chain loader, wrap that first program even if its main code is small. Never run a target of `CHAIN` or `SDC_CHAIN` through `cc1sl`. The chain command is itself the 64K loader; it expects the compiler's normal DECB LOADM file and loads every segment before jumping to the execution address in that file's postamble.

Both chain loaders force the computer into all-RAM mode before copying the target, so the target may contain segments and executable code above `$7FFF` even if the calling program entered `CHAIN` while the BASIC ROMs were selected. The floppy loader temporarily selects ROM mode only during physical disk-controller transfers so the CoCo 1/2 hardware NMI path remains available; it returns to all-RAM mode before processing the loaded bytes.

Example build sequence

Suppose `START.BAS` launches `PART2.BIN`:

```
CHAIN "PART2.BIN:0"
```

Compile and assemble both programs normally. Use `cc1sl` only on the first program:

```
./BasTo6809 START.BAS
lwasm -9bl -p cd -oSTART.BIN START.asm
./cc1sl -l START.BIN -oGO.BIN

./BasTo6809 PART2.BAS
lwasm -9bl -p cd -oPART2.BIN PART2.asm
```

Load and run `GO.BIN` from Disk BASIC. `GO.BIN` starts the first program; its `CHAIN` command then loads the unwrapped `PART2.BIN` and transfers control to it. The same rule applies when the first program uses:

```
SDC_CHAIN "PART2.BIN",0
```

In that case, copy the normal unwrapped `PART2.BIN` to the CoCoSDC filesystem.

Preparing every chained program

- Compile every program in the sequence with the same `-notext` setting. This keeps the resident-loader, graphics, and program layouts compatible.
- A program only includes the resident loader for the chain commands it actually uses. If `PART2.BIN` will later execute `CHAIN` or `SDC_CHAIN`, include that command in `PART2.BAS` so the compiler adds the required loader to `PART2.BIN`.
- Use `-rxxxx` consistently when programs exchange data through reserved shared RAM. Each program must agree on the address and format of that shared data.
- A chain target must be a valid DECB LOADM `.BIN` with an execution address in its postamble. Do not use the `cc1sl` output as a chain target.
- `CHAIN` and `SDC_CHAIN` do not return. Save any state needed by the next program before executing the command.

Compiling for a Dragon computer

There are a few things you need to do in order to compile and run a program on a Dragon computer. Since the Dragon computer is very similar to a CoCo 1 code from the compiler that works on a CoCo 1 & 2 will pretty much execute as is on a Dragon computer. But the keyboard layout is different between the two computers. When compiling your program to target a Dragon computer you must use the -dragon option. This tells the compiler to include the proper keyboard routines that will work for the Dragon computer. Also since the Dragon uses a different file format for loading binary files you must copy the DRAGLOAD.BIN file to your floppy disk or emulator disk image. You will also need to rename your compiled program to COMPILED.BIN and copy COMPILED.BIN to the same floppy disk or emulator disk image as DRAGLOAD.BIN. You can rename DRAGLOAD.BIN

The steps you would take to compile a program called HELLO.BAS

```
1) Compile your program into .asm file
./BasTo6809 -ascii -dragon HELLO.BAS
2) Assemble the program into a DECB machine language binary file
lwasm -9b -p cd -o./HELLO.bin HELLO.asm
3) Copy the DRAGLOAD.BIN file to a floppy disk or disk image and rename the file as
HELLO.BIN
./decb copy -2 -b -r ./DRAGLOAD.BIN DISK1.DSK, HELLO.BIN
4) Copy your actual program HELLO.bin to a floppy disk or disk image, and rename the
file as COMPILED.BIN
./decb copy -2 -b -r ./HELLO.bin DISK1.DSK,COMPILED.BIN
```

Your disk should now have two files on it called HELLO.BIN (which is really the DRAGLOAD.BIN program) and COMPILED.BIN (which is your actual compiled program). The loader expects the file to be loaded from drive 1 and you start your program on the Dragon computer by typing: RUN"HELLO.BIN"

The DRAGLOAD.BIN program will handle programs that are 64k.

BASIC commands

ATTR c1,c2,B,U

Sets display attributes of 40,64 and 80 column text screen. c1 Foreground Palette # (0 to 7) c2 Background Palette # (0 to 7) B Character blink on U Underline on

EXAMPLE

```
ATTR 3,2,U
Add 8 to the Foreground value to match the real palette
```

AUDIO switch

Connects or disconnects cassette audio output to the display speaker. switch ON Routes sound from cassette player to the display speaker switch OFF Disconnects cassette player sound from the display speaker

EXAMPLE

```
AUDIO OFF
```

CIRCLE(x,y),Radius,Colour

Draws a circle on the screen To keep the aspect ratio close to round some of the graphics modes use scaling so the Radius isn't always a count of actual pixel values.

- x,y - Origin of the circle
- Radius - Size of the circle
- Colour - Colour Number of the circle to draw

EXAMPLE

```
CIRCLE (X,Y),R,C
```

CLEAR

Erases all variables and arrays

EXAMPLE

```
CLEAR
```

CLS [c]

Clears the text screen to a specified colour. If c is not specified, uses the current background colour. c Colour code

EXAMPLE

```
CLS 2
```

CMP

Resets the palette registers to the standard colours for a composite monitor/TV.

EXAMPLE

```
CMP
```

COLOR c1,c2

Sets the foreground and background colours of the current low-resolution graphics screen. c1 Foreground colour code (0–8) c2 Background colour code (0–8)

EXAMPLE

```
COLOR 2,3
```

```
CONST constantName = value[, ...]
```

The CONST statement globally defines one or more named numeric or string values which will not change while the program is running.

EXAMPLE

```
CONST PI = 3.141592654
```

```
COPYBLOCKS source,destination,#
```

This is a CoCo 3 specific command which copies 8k blocks to other 8k blocks very fast. Useful for double buffering CoCo 3 background screens

- source - First source block to be copied from
- destination - First Destination block to be copied to

- **- Number of blocks to copy**

EXAMPLE

```
COPYBLOCKS &H10,&H20,3
```

```
CPUSPEED #
```

The compiler will detect and run the CPU at the maximum speed it can on the hardware it is running. If you want to change the speed use this command to set the CPU speed to value x at 1.79 Mhz at 2.864 MHz If # is anything else then the CPU will be set in Native mode and run at it's max speed

- **= 1 Set the CPU in Emulation mode and set the speed at .895 Mhz**

- **= 2 Set the CPU in Emulation mode and set the**

speed

- ## = 3 Set the CPU in Emulation mode and set the speed

EXAMPLE

```
CPUSPEED 1
```

DATA constant,constant,...

Stores numeric and string constants for use with the READ statement. constant String or numeric constant(s), such as 127, 2985, or BEAGLE

EXAMPLE

```
DATA 45,"CAT",98,"DOG",24.3,1000
```

DIM variable AS type

Declares a variable or array with a specific numeric or string type. Use DIM A$(3,10), R4(22) to create a string array and a SINGLE array. Use DIM R4(10) AS _BYTE, X AS LONG to choose explicit numeric types. A fixed maximum string length uses QB64PE-compatible syntax: DIM NAME$ AS STRING * 32. Assignments longer than the declared maximum are truncated; the normal string-variable naming rules remain unchanged.

EXAMPLE

```
DIM NAME$ AS STRING * 32
```

DO ... LOOP

Defines a loop that repeats a block of statements. You can control it with DO WHILE/UNTIL condition and/or LOOP WHILE/UNTIL condition.

EXAMPLE

```
Do While A < 10
A = A + 1
Loop
Do Until A > 9
A = A + 1
Loop
Do
A = A + 1
Loop Until A = 10
Do
A = A + 1
Loop While A < 10
```

DRAW string

Draws a line on the current graphics screen as specified by string. Works in all Gmode screens. String commands include: A Angle BM Blank move C Color D Down E 45-degree angle F 135-degree angle G 225-degree angle H 315-degree angle L Left M Move draw position N No update R Right S Scale U Up

EXAMPLE

```
DRAW "BM128,96;U25;R25;D25;L25"
```

END

Marks the end of a BASIC program.

EXAMPLE

```
END
```

EXEC (address)

Transfers control to a machine-language program at address.

EXAMPLE

```
EXEC &H7022
```

EXIT DO / EXIT FOR / EXIT WHILE

Exits the current DO...LOOP, FOR...NEXT, or WHILE... WEND loop immediately; execution continues with the first statement after the loop's ending keyword (LOOP, NEXT, or WEND).

EXAMPLE

```
WHILE A < 100
IF A = 10 THEN
EXIT WHILE
ELSE
A = A + 1
END IF
WEND
```

FOR variable=n1 TO n2 STEP n3

Defines the beginning of a loop. The end is specified by NEXT. variable Loop counter variable n1 Starting value of counter n2 Ending value of counter n3 Increment or decrement value (optional; default is 1)

EXAMPLE

```
FOR Z=35 TO 125 STEP 5
```

GCLS [c]

Colour the graphics screen c is the colour value you want the screen to be coloured

EXAMPLE

```
GCLS 2
```

GCOPY SourcePage,DestinationPage

Makes a copy the Source graphics page to the Destination graphics page.

- SourcePage - Source graphics page

- DestinationPage - Destination graphics page

EXAMPLE

```
GCOPY 0,3
```

GETJOYD

Quick Read of both joysticks (horizontal and vertical) as digital values (0, 31, or 63) and stores the results in the standard BASIC joystick locations: Left V/H at $015A/$015B Right V/H at $015C/$015D

EXAMPLE

```
GETJOYD
```

GMODE ModeNumber,GraphicsPage

Selects the graphics screen and the graphics page. ModeNumber is the graphics mode you want to use GraphicsPage is the Page you want to show/use for your graphics commands

To see a list of ModeNumbers and the resolutions go here **Special note the ModeNumber must be an actual number and cannot be a variable as the compiler needs to know exactly which graphic mode commands need to be included at compile time. GraphicsPage can be a variable. If you are going to use Graphic pages, the compiler needs to know how many pages to reserve in RAM (for CoCo 1 & 2 graphics). So you must have a GMODE #,MaxPages entry at the beginning of your BASIC program. Where the value of MaxPages will be an actual number and not a variable.

EXAMPLE

```
GMODE 16,0
** Special note if you are going to use GMODE 160 to 165
(the special NTSC composite 256 colour modes).  When the
compiler comes across the first GMODE command with the
values of 160 to 165 it will automatically set the palette to the
following values:
Palette 0,0  ' xx000000
Palette 1,16 ' xx010000
Palette 2,32 ' xx100000
Palette 3,48 ' xx110000
```

GOSUB line

Calls a subroutine beginning at the specified line number or label

EXAMPLE

```
GOTO Complete
...
Complete:
```

GOTO line or label

Jumps to the specified line number or label

EXAMPLE

```
GOTO Complete
...
Complete:
```

IF test THEN ... ELSEIF ... ELSE ... END IF

Multi-line decision block. BASIC runs the first true section; if none are true, it runs the ELSE section (if present).

EXAMPLE

```
IF X=0 THEN
PRINT "ZERO"
ELSEIF X<0 THEN
PRINT "NEG"
ELSE
PRINT "POS"
END IF
```

INPUT var1,var2,...

Reads data from the keyboard and stores it in one or more variables.

EXAMPLE

```
INPUT K3
```

LET

Assigns a value to a variable (optional keyword).

EXAMPLE

```
LET A3=27
```

LINE(x0,y0)-(x1,y1),Colour[,B][F]

- x0,y0 - Starting location
- x1,y1 - Ending location
- Colour - Colour of the Line or Box to draw
- B - Draw a Box
- F - Fill the Box

EXAMPLE

```
LINE (22,33)-(27,39),1,BF
```

LOOP

LOOP is used in a DO/LOOP, see DO for more information.

LOCATE x,y

Moves the high-resolution text screen cursor to position x,y.

EXAMPLE

```
LOCATE 20,12
```

LPOKE location,value

Stores a value (0–255) in a virtual memory location (0-2097151 decimal or 0-&H1FFFFF hex)

EXAMPLE

```
LPOKE 480126,241
```

MOTOR

Turns the cassette motor ON or OFF.

EXAMPLE

```
MOTOR ON
```

NEXT v1,v2,...

Defines the end of a FOR loop. v1,v2,... Optional variable names for nested loops (if used, list in reverse order of the FOR variables). If omitted, ends the most recent FOR.

EXAMPLE

```
NEXT X,Y,Z
```

NTSC_FONTCOLOURS b,f

Sets the background (b) and foreground (f) colours used by the NTSC composite-output fonts in GMODE 160 to 165.

EXAMPLE

```
NTSC_FONTCOLOURS 0,3
```

ON expr GOSUB line1,line2,...

Multiway call to one of several subroutines, based on expr.

EXAMPLE

```
ON A GOSUB 100,230,500,1125
```

ON expr GOTO line1,line2,...

Multiway branch to one of several line numbers, based on expr.

EXAMPLE

```
ON A GOTO 100,230,500,1125
```

PAINT(x,y),OldColour,FillColour

Fills the old colour value with the fill colour value which must also be the border colour of the section you are painting

- x,y - Starting location
- OldColour - Colour Number
- FillColour - Colour Number

EXAMPLE

```
PAINT (44,55),2,3
```

PALETTE pr,c

Stores colour code c (0–63) into palette register pr (0–15) directly.

EXAMPLE

```
PALETTE 1,13
This command is executed immediately
```

PALETTEW pr,c

Stores colour code c (0–63) into palette register pr (0–15) Mirror.

EXAMPLE

```
PALETTEW 1,13
This command does not update the palette register until the
PALETTEV command is used.  It allows the user to update a
bunch of palette registers then when vsync occurs it will
change the hardware palette registers.
```

PALETTEV pr,c

Stores colour code c (0–63) into palette register pr (0–15) Mirror, Waits for a VSYNC then it copies the values in the Palette register Mirror to the hardware palette registers.

EXAMPLE

```
PALETTE 1,13
```

PLAY string

Plays music as specified by string. Common commands include: A–G Notes L Length O Octave P Pause T Tempo # or + Sharp - Flat

EXAMPLE

```
PLAY "L1;A;A#;A-"
```

PLAYFIELD

Sets the scrollable playfield mode in your BASIC program. The # must be a literal number, not a variable.

To create playfield graphics, use the command-line tool PNGtoCC3Playfield to convert a PNG into a file (or files). These can then be loaded into memory using one of the following commands in BASIC: - LOADM - SDC_LOADM - SDC_BIGLOADM

Once loaded, the background playfield becomes scrollable according to the selected mode. # Max Resolution Min Resolution Size Multiple

- 1 - 256 x 7872 256 x 192 not applicable x 64
- 2 - 512 x 3840 512 x 192 not applicable x 192
- 3 - 1024 x 512 1024 x 256 not applicable x 256
- 4 - 2048 x 256 512 x 256 256 x not applicable
- 5 - 2560 x 192 512 x 192 256 x not applicable

POKE location,value

Stores a value (0–255) in a memory location (0–65535 decimal or 0–$FFFF hexadecimal).

EXAMPLE

```
POKE 28000,241
```

PRINT message

Prints on the text screen.

EXAMPLE

```
PRINT "HELLO!"
```

PRINT #-3,

Print to the graphics screen created with the GMODE command. The default font is Arcade_B0_F1, but you can select others using the compiler command -fxxxx Where xxxx is the font name used for printing to the graphics screen (default is Arcade_B0_F1). Look in folder Basic_Includes/GraphicsMode/Graphic_Screen_Fonts to see the font names available

PRINT #-4, and PRINT #-5,

Special print commands that prints on the GMODE 16 screen with a dedicated small font that is 42 columns x 24 rows using 6x8 one-bit characters on the 256x192 screen. #-4, prints with a black background and white foreground #-5, print with a white background and a black background

PRINT TAB(n)

Moves the cursor to column n on the low- and high- resolution text screens.

EXAMPLE

```
PRINT TAB(22);"HELLO!"
PRINT@ n,message
Prints message on the low-resolution text screen at position
n.
PRINT@11,"HELLO!"
```

READ var1,var2,...

Reads the next item(s) from DATA statements and stores them into variables. forever reading from RAM if it reaches the end of DATA.

EXAMPLE

```
READ A1,B,C7
```

> **Note:** Make sure to limit READ commands as the compiler will go

REM comment

Inserts a remark (comment). BASIC ignores everything after REM on that line.

EXAMPLE

```
REM THIS IS A COMMENT LINE
```

RESTORE

Resets the READ pointer back to the first item on the first DATA line.

EXAMPLE

```
RESTORE
```

RETURN

Returns from a subroutine to the statement following the last GOSUB.

EXAMPLE

```
RETURN
```

RGB

Resets the palette registers to the standard colours for an RGB monitor.

EXAMPLE

```
RGB
```

RUN

Executes a program.

EXAMPLE

```
RUN
```

SAMPLE option (For CoCo 3 only)

Control audio sample playback. The following options are available:

- SINGLE #- Plays sample # once
- LOOP # - Plays sample # continuously
- OFF - Turns off sample playback

EXAMPLE

```
SAMPLE SINGLE 2
```

SAMPLE_LOAD "filename", sample number

Loads a raw audio sample (samples must be raw 8 bit unsigned at 6003 hz) into the CoCo memory and assigns a number to the sample to be referenced for play back with the SAMPLE command.

EXAMPLE

```
SAMPLE_LOAD "JUMP.raw", 0
```

SCREEN type,colours

Selects screen modes and colour sets. type: 0 Text 1 Graphics colours: 0 Colour set 0 1 Colour set 1

EXAMPLE

```
SCREEN 0,1
```

SELECT CASE expr ... CASE ... CASE ELSE ... END SELECT

Multi-way branch based on expr. BASIC executes the first matching CASE block; if none match, it executes CASE ELSE (if present).

EXAMPLE

```
Select Case A
Case 1
Print "ONE"
Case 2
Print "TWO"
Case Else
Print "OTHER"
End Select
```

SET(x,y,Colour)

Sets a pixel on the screen

- x,y - Screen location of the pixel to be drawn
- Colour - Colour Number of the pixel to be drawn

EXAMPLE

```
SET (11,11,3)
```

SLEEP x

Used to pause program execution continuing again.

- x - Number of milliseconds to pause the program before

SOUND tone,duration

Sounds a tone for a specified duration. tone 1–255 sets pitch duration 1–255 sets duration

EXAMPLE

```
SOUND 33,22
```

SPRITE (options)

Controls sprites on the screen (turn on/off, position, draw/ erase, and background save/restore). Sprites are referenced by sprite number #, with optional frame f for animated sprites.

EXAMPLE

```
SPRITE LOCATE 1, 80, 60 : SPRITE SHOW 1, 2
Sprite Option details:
SPRITE OFF [#]
Turns off sprite number #.
If no number is provided, all sprites are turned off.
SPRITE LOCATE #, x, y
Moves sprite # to a new position on the screen.
x and y are the playfield coordinates.
SPRITE SHOW #[, f]
Displays sprite # using frame f.
If the sprite is not animated (i.e. it has only one frame) use of   ,f is optional.
SPRITE BACKUP #
Saves the background behind sprite #.
This allows the sprite to be cleanly erased later.
SPRITE ERASE #
Erases sprite # and restores the background previously
saved with SPRITE BACK.
SPRITE #,x,y[,f]
Draws sprite number # at x,y co-ordinates using frame f.
f is optional, a value of 0 (single frame sprite) is used if not
given.
```

SPRITE_LOAD "SpriteName.asm", # [,f]

Loads a sprite definition from an assembly-format sprite file (Generated by the command line tool called PNGtoCCSprite) and assigns it to sprite number # for later use by SPRITE commands. Optional f specifies the number of frames (omit if the sprite has only one frame).

EXAMPLE

```
SPRITE_LOAD "HERO.asm", 1, 4
```

STOP

Stops execution of a program.

EXAMPLE

```
STOP
TIMER=n
Sets TIMER to n.
TIMER=120
```

UNTIL

UNTIL is also used in a DO/LOOP, see DO for more information.

VIEW x,y

Selects which part of the scrollable playfield is shown on-screen. The x and y values specify the top-left corner of the visible window into the playfield.

EXAMPLE

```
VIEW 0,0
```

WAIT VBL

Wait for Vertical Blank then update the sprites on screen This command requires sprites to be setup in the program

EXAMPLE

```
Wait VBL
```

WHILE test ... WEND

Repeats a block of statements while the test is true. When the test becomes false, execution continues with the line after WEND. WHILE is also used in a DO/LOOP, see DO for more information.

EXAMPLE

```
While A < 10
A = A + 1
Wend
```

WIDTH n

Sets the text screen resolution: 32 32x16 (low-resolution text) 40 40x28 (high-resolution text) 64 64x28 (high-resolution text) 80 80x28 (high-resolution text)

EXAMPLE

```
WIDTH 80
```

Disk Related Commands

> These commands access standard DECB floppy disks. Drive suffixes use :0 through :3; drive 0 is used when the suffix is omitted. The sequential interface supports one reader and one writer at the same time when they use different file numbers.

```
x = _FILEEXISTS(string)
```

Returns 255 (-1) when the named floppy-disk file exists, or 0 when it does not. The filename may include an optional drive suffix.

EXAMPLE

```
A = _FILEEXISTS("DATA.DAT:1")
```

```
CHAIN "FILENAME.BIN[:drive]"
```

Loads a DECB machine-language program from floppy disk, replaces the current program, and executes the address in the LOADM postamble. The drive number is optional. The .BIN extension is added when no extension is supplied. CHAIN does not return to the original program.

The target must be the normal `.BIN` produced by assembling the compiler's output. Do not process a program that will be loaded by `CHAIN` with `cc1sl`; `CHAIN` already contains the loader needed to load a large program across the 64K address space. A compiler-generated first program containing `CHAIN` should be processed with `cc1sl` when Disk BASIC loads it, even if its main code is small, because the resident loader occupies part of Disk BASIC's loading workspace. See Preparing programs for CHAIN and SDC_CHAIN for the complete build procedure.

EXAMPLE

```
CHAIN "PART2.BIN:1"
```

```
OPEN "FILENAME.EXT[:drive]","X",#
```

Opens a DECB disk file for sequential byte access. Use R to read an existing file or W to create or replace a file. # is file number 0 or 1. A reader and writer may be open together, but they must use different file numbers.

EXAMPLE

```
OPEN "DATA.DAT:2","R",0
```

CLOSE(#)

Closes file number 0 or 1. Closing an output file flushes buffered data and commits the FAT and directory updates. An output file must contain at least one byte before it is closed. The disk motor is turned off automatically after disk activity stops.

EXAMPLE

```
CLOSE(0)
```

PUTBYTE0 x

Writes the low eight bits of x to output file 0. The file position advances automatically by one byte.

EXAMPLE

```
PUTBYTE0 65
```

PUTBYTE1 x

Writes the low eight bits of x to output file 1. The file position advances automatically by one byte.

EXAMPLE

```
PUTBYTE1 ASC("A")
```

x = GETBYTE(#)

Reads and returns one byte from input file 0 or 1. The file position advances automatically. GETBYTE returns 0 after end-of-file; because zero is also a valid byte, use LOF to control loops that must read every byte.

EXAMPLE

```
B = GETBYTE(0)
```

x = LOF(#)

Returns the length of open file 0 or 1 as an unsigned 32-bit value (_UNSIGNED LONG). Input files report their complete length; output files report the bytes written so far, including buffered bytes.

EXAMPLE

```
FileSize = LOF(0)
```

SETPOS0(x)

Sets file 0 to zero-based unsigned 32-bit byte offset x. For input, this only changes the read position and never writes data; seeking beyond end-of-file stops at the end. For output, seeking forward fills skipped bytes with zeros. Seeking backward in an output file produces a runtime error.

EXAMPLE

```
SETPOS0(299)
```

SETPOS1(x)

Sets file 1 to zero-based unsigned 32-bit byte offset x. For input, this only changes the read position and never writes data; seeking beyond end-of-file stops at the end. For output, seeking forward fills skipped bytes with zeros. Seeking backward in an output file produces a runtime error.

EXAMPLE

```
SETPOS1(0)
```

A$ = FILEINFO$(#)

Returns a 32-byte record for open file 0 or 1. Bytes 1-8 contain the space-padded name; 9-11 the extension; byte 12 the attributes (zero for DECB); 13-28 are reserved; and 29-32 contain the unsigned 32-bit size, least-significant byte first. With fixed-length strings, A$ must hold at least 32 bytes.

EXAMPLE

```
A$ = FILEINFO$(0)
```

x = DELETE(A$)

Deletes the named DECB file. A$ may include a drive suffix. No sequential input or output file may be open. Returns 0 when deleted and 5 when not found; other disk failures produce a runtime disk error.

EXAMPLE

```
R = DELETE("OLD.DAT:1")
```

x = INITDIR(A$)

Initializes a DECB directory listing. Call it before DIRPAGE. The pattern supports * for the rest of a filename field and ? for one character; use . for all files. A drive suffix is optional. No sequential file may be open. Returns 0 on success; disk failures produce a runtime disk error.

EXAMPLE

```
R = INITDIR("*.DAT:0")
```

x = DIRPAGE(0)

Gets the next directory page, containing up to sixteen records. Read the records with DIRLIST$(0) through DIRLIST$(15). Unused records on the last partial page contain sixteen zero bytes. Returns 0 for a retrieved page, including a final partial or empty page, and 4 when no listing is initialized or all 72 directory slots have been scanned.

EXAMPLE

```
R = DIRPAGE(0)
```

A$ = DIRLIST$(y)

Returns record y (0 through 15) from the most recent DIRPAGE. Bytes 1-8 are the filename, 9-11 the extension, byte 12 the attribute ($20 for a regular file), and 13-16 the unsigned 32-bit size in most-significant-byte-first order. An unused record contains sixteen zero bytes. With fixed-length strings, A$ must hold at least sixteen bytes.

EXAMPLE

```
A$ = DIRLIST$(3)
```

LOADM "filename.ext[:drive]"[,offset]

Loads a DECB machine-language file from floppy disk. The optional offset is added to every segment's load address. Files may contain more than 64 KB in total when composed of multiple segments; each segment address and length remains 16-bit.

EXAMPLE

```
LOADM "PROG.BIN:2",4231
```

SAVEM "filename.ext[:drive]",Start,End,Exec

Saves a memory range to a DECB floppy disk as a machine-language file. Start and End define the inclusive range; Exec is the run address.

EXAMPLE

```
SAVEM "LEVEL1.BIN:1",28000,29000,28000
```

Function Commands

ABS (n)

Returns the absolute value of n.

EXAMPLE

```
A = ABS(B)
```

ASC (string)

Returns the code (ASCII value) of the first character in string.

EXAMPLE

```
A = ASC(B$)
```

ATN (n)

Returns the arctangent of n, in radians.

EXAMPLE

```
A = ATN(B/3)
```

BUTTON (n)

Returns 1 if joystick button n is being pressed; returns 0 if it is not. n can be:

- 0 - Right joystick, Button 1 (old joystick)
- 1 - Right joystick, Button 2
- 2 - Left joystick, Button 1 (old joystick)
- 3 - Left joystick, Button 2

EXAMPLE

```
A = BUTTON(D)
```

CHR$ (n)

Returns the character corresponding to character code n.

EXAMPLE

```
A$ = CHR$(65)
```

COCOHARDWARE(0)

Returns with a numeric value representing which CoCo you are using.

EXAMPLE

```
V = COCOHARDWARE(0)
Where the bits of variable V will signify the CoCo
Hardware as:
Bit 0 is the Computer Type,0 = CoCo 1/2, 1 = CoCo 3
Bit 7 is the CPU type,         0 = 6809, 1 = 6309
```

COS (angle)

Returns the cosine of angle (angle is in radians).

EXAMPLE

```
A = COS(B)
```

EXP (n)

Returns the natural exponential of n (e^n).

EXAMPLE

```
A = EXP(B*1.15)
```

FIX (n)

Returns the truncated integer of n. Unlike INT, FIX does not return the next lower number for negative values; it truncates toward zero.

EXAMPLE

```
A = FIX(B - .2)
```

HEX$ (n)

Returns a string containing the hexadecimal value of n.

EXAMPLE

```
PRINT HEX$(A); "="; A
```

INKEY$

Checks the keyboard and returns the key being pressed or, if no key is being pressed, returns a null string ("").

EXAMPLE

```
A$ = INKEY$
```

INSTR (p, s, t)

Searches a string. Returns the position of target string t, within search string s, starting at position p.

- p - Start position of search
- s - String being searched
- t - Target string

EXAMPLE

```
A = INSTR(1, M5$, "BEETS")
```

INT (n)

Converts n to the largest integer that is less than or equal to n.

EXAMPLE

```
A = INT(B + .5)
```

JOYSTK (i)

Returns the horizontal or vertical coordinate (i) of the left or right joystick.

- 0 - Horizontal, right joystick
- 1 - Vertical, right joystick
- 2 - Horizontal, left joystick
- 3 - Vertical, left joystick

EXAMPLE

```
A = JOYSTK(B)
```

LEFT$ (string, length)

Returns the left portion of a string. length specifies the number of characters returned.

EXAMPLE

```
A$ = LEFT$(B$, 3)
```

LEN (string)

Returns the length of string.

EXAMPLE

```
A = LEN(B$)
```

LOG (n)

Returns the natural logarithm of n.

EXAMPLE

```
A = LOG(B / 2)
```

LPEEK (memory location)

Returns the contents of a virtual memory location (0-2097151 decimal or 0-&H1FFFFF hex)

EXAMPLE

```
A = LPEEK(&H60000)
```

LTRIM$ (string)

Removes both leading spaces from the string.

EXAMPLE

```
A$ = LTRIM$(B$)
```

MID$ (s, p, l)

Returns a substring of string s.

- s - Source string
- p - Starting position of substring
- l - Length of substring

EXAMPLE

```
A$ = MID$(B$, 2, 2)
```

PEEK (memory location)

Returns the contents of a memory location (0–65535 decimal or 0–&HFFFF hexadecimal).

EXAMPLE

```
A = PEEK(3020)
```

POINT (x,y)

Returns the colour value of the pixel selected from the current GMODE screen.

- x,y - Screen location of the pixel value requested

EXAMPLE

```
A = POINT(10,12)
```

POS (dev)

Returns the current print position. dev (print device number): -2 - Printer

- 0 - Screen

EXAMPLE

```
A = POS(0)
```

RIGHT$ (string, length)

Returns the right portion of a string. length specifies the number of characters returned.

EXAMPLE

```
A$ = RIGHT$(B$, 4)
```

RND (n)

Generates a "random" number between 1 and n. If n = 0 then it generates a random number from 0 to <1

EXAMPLE

```
A = RND(0)
```

RTRIM$ (string)

Removes trailing spaces from the string.

EXAMPLE

```
A$ = RRIM$(B$)
```

SGN (n)

Returns the sign of n: -1 - Negative

- 0 - Zero
- 1 - Positive

EXAMPLE

```
A = SGN(A + .1)
```

SIN (angle)

Returns the sine of angle (angle is in radians).

EXAMPLE

```
A = SIN(B/3.14159)
```

STRING$ (l, c)

Returns a string of a repeated character.

- l - Length of string
- c - Character used (can be a code or a string)

EXAMPLE

```
A$ = STRING$(22, "A")
```

STR$ (n)

Converts n to a string.

EXAMPLE

```
A$ = STR$(1.234)
```

SQR (n)

Returns the square root of n.

EXAMPLE

```
A = SQR(B/2)
```

TAN (angle)

Returns the tangent of angle (angle is in radians).

EXAMPLE

```
A = TAN(B)
```

TIMER

Returns the contents of the timer (0–65535).

EXAMPLE

```
A = TIMER / 18
```

TRIM$ (string)

Removes both leading and trailing spaces from the string.

EXAMPLE

```
A$ = TRIM$(B$)
```

VAL (string)

Converts a string to a number.

EXAMPLE

```
A = VAL("1.23")
```

VARPTR (variable)

Returns a pointer to where a variable is located in memory.

EXAMPLE

```
A = VARPTR(B)
You cannot change where a variable is in memory, you can
only use this to find out where it exists.
```

Constants

You can use Constants in your program that will act like place holders for actual values that will be substituted at compile time. They are different than variables as they don't actually take any RAM in your program.

```
Examples:
Const PI = 3.141593
Const PI2 = PI * 2
```

Constants can also be quoted strings:

```
Const circumferenceText = "THE CIRCUMFERENCE OF THE CIRCLE IS"
Const areaText = "THE AREA OF THE CIRCLE IS"
```

Math / Logical Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division (Integers will be rounded)
^	Exponent
MOD	Remainder of division
\	Integer division (no rounding)
AND	Bitwise AND
OR	Bitwise OR
XOR	Bitwise Exclusive OR
NOT	Bitwise Compliment (invert each bit)

CoCoMP3 Commands

> You can easily and directly control the CoCoMP3 hardware from your BASIC program using the following commands:

COCOMP3_AUDIO_MODE(x)

Sets where CoCoMP3 audio is routed. x=0 leave unchanged x=1 (default) route audio to the CoCo cassette interface so it plays through the TV speaker.

EXAMPLE

```
z=COCOMP3_AUDIO_MODE(1)
```

COCOMP3_COMBINATION_PLAY_SETTING(x,y)

Sets a "combination play" range of tracks from x to y.

EXAMPLE

```
z=COCOMP3_COMBINATION_PLAY_SETTING(10,25)
```

COCOMP3_CYCLE_MODE_SETTING(x)

Sets playback mode. x=0 full cycle 1 repeat track 2 stop after track 3 random all 4 repeat folder 5 random in folder 6 order in folder then stop 7 order all then stop.

EXAMPLE

```
z=COCOMP3_CYCLE_MODE_SETTING(4)
```

COCOMP3_END_COMBINATION_PLAY(0)

When a combination play range finishes, stop playback.

EXAMPLE

```
z=COCOMP3_END_COMBINATION_PLAY(0)
```

COCOMP3_END_PLAYING(0)

Ends the current track and skips to the next track (like Next).

EXAMPLE

```
z=COCOMP3_END_PLAYING(0)
```

COCOMP3_GET_CURRENT_TRACK(0)

Returns the current track number assigned by the CoCoMP3 (usable with PLAY_TRACK_NUMBER or SELECT_BUT_NO_PLAY).

EXAMPLE

```
z=COCOMP3_GET_CURRENT_TRACK(0)
```

COCOMP3_GET_DRIVE_STATUS(0)

Returns microSD status:

- 0 = not installed
- 2 = installed

EXAMPLE

```
z=COCOMP3_GET_DRIVE_STATUS(0)
```

COCOMP3_GET_FOLDER_DIR_TRACK(0)

Returns the first track number in the current folder.

EXAMPLE

```
z=COCOMP3_GET_FOLDER_DIR_TRACK(0)
```

COCOMP3_GET_NUMBER_OF_TRACKS(0)

Returns the total number of tracks on the microSD.

EXAMPLE

```
z=COCOMP3_GET_NUMBER_OF_TRACKS(0)
```

COCOMP3_GET_PLAY_STATUS(0)

Returns play status:

- 0 = stopped
- 1 = playing

EXAMPLE

```
z=COCOMP3_GET_PLAY_STATUS(0)
```

COCOMP3_GET_TRACKS_IN_FOLDER(0)

Returns how many tracks are in the current folder.

EXAMPLE

```
z=COCOMP3_GET_TRACKS_IN_FOLDER(0)
```

COCOMP3_NEXT(0)

Skips to (and plays) the next track.

EXAMPLE

```
z=COCOMP3_NEXT(0)
```

COCOMP3_PAUSE(0)

Pauses playback of the current track.

EXAMPLE

```
z=COCOMP3_PAUSE(0)
```

COCOMP3_PLAY(0)

Plays the current track (or the first track after power-on).

EXAMPLE

```
z=COCOMP3_PLAY(0)
```

COCOMP3_PLAY_NEXT_FOLDER(0)

Advances to the next folder and starts playing there (behavior depends on Cycle Mode).

EXAMPLE

```
z=COCOMP3_PLAY_NEXT_FOLDER(0)
```

COCOMP3_PLAY_PREVIOUS_FOLDER(0)

Moves to the previous folder and starts playing there (behavior depends on Cycle Mode).

EXAMPLE

```
z=COCOMP3_PLAY_PREVIOUS_FOLDER(0)
```

COCOMP3_PLAY_TRACK(T$)

Selects and plays a specific file path. The command converts ".MP3/.WAV" to "*MP3/*WAV" as required by the hardware.

EXAMPLE

```
z=COCOMP3_PLAY_TRACK("/FOLDER02/00001.MP3")
```

COCOMP3_PLAY_TRACK_NUMBER(x)

Plays track number x (track numbers follow copy order to the microSD; use GET_CURRENT_TRACK to discover a track's number).

EXAMPLE

```
z=COCOMP3_PLAY_TRACK_NUMBER(12)
```

COCOMP3_PREVIOUS(0)

Skips to (and plays) the previous track.

EXAMPLE

```
z=COCOMP3_PREVIOUS(0)
```

COCOMP3_RAW(R$)

Sends a raw command packet to the CoCoMP3 (for advanced control; see DY-SV5W_Datasheet.pdf for codes).

EXAMPLE

```
z=COCOMP3_RAW(CHR$(&HAA)+CHR$(1)+CHR$(0)+CHR$(&HAB))
```

COCOMP3_SELECT_BUT_NO_PLAY(x)

Stops the current track, queues track x, but does not start playback until you call PLAY.

EXAMPLE

```
z=COCOMP3_SELECT_BUT_NO_PLAY(12)
```

COCOMP3_SET_CYCLE_TIMES(x)

Sets how many times to repeat the current track/folder before stopping; manually selecting another track overrides this.

EXAMPLE

```
z=COCOMP3_SET_CYCLE_TIMES(3)
```

COCOMP3_SET_EQ(x)

Sets the internal EQ preset: 0 Normal 1 Pop 2 Rock 3 Jazz 4 Classical

EXAMPLE

```
z=COCOMP3_SET_EQ(2)
```

COCOMP3_SET_PATH_INTERLUDE(P$)

Interrupts the current track, plays tracks from folder path P$, then resumes the interrupted track afterward (behaviour may depend on Cycle Mode).

EXAMPLE

```
z=COCOMP3_SET_PATH_INTERLUDE("/FOLDER02/*MP3")
```

COCOMP3_SET_TRACK_INTERLUDE(x)

Interrupts the current track, plays track x, then resumes the interrupted track afterward.

EXAMPLE

```
z=COCOMP3_SET_TRACK_INTERLUDE(20)
```

COCOMP3_SET_VOL(x)

Sets volume level 0–30 (default on power-up is 30).

EXAMPLE

```
z=COCOMP3_SET_VOL(18)
```

COCOMP3_STOP(0)

Stops playback of the current track.

EXAMPLE

```
z=COCOMP3_STOP(0)
```

COCOMP3_TEST(0)

Checks if CoCoMP3 is ready. Returns: 0 OK -1 not powered/connected -2 no microSD -3 no tracks on microSD.

EXAMPLE

```
z=COCOMP3_TEST(0)
```

COCOMP3_VOL_DOWN(0)

Decreases volume by 1 step.

EXAMPLE

```
z=COCOMP3_VOL_DOWN(0)
```

COCOMP3_VOL_FADE(x)

Fades volume to 0 over x milliseconds, then stops playback (software-driven; the CoCo is halted during the fade).

EXAMPLE

```
z=COCOMP3_VOL_FADE(1500)
```

COCOMP3_VOL_MAX(0)

Sets volume to maximum (30).

EXAMPLE

```
z=COCOMP3_VOL_MAX(0)
```

COCOMP3_VOL_UP(0)

Increases volume by 1 step.

EXAMPLE

```
z=COCOMP3_VOL_UP(0)
```

CoCo SDC Specific Commands

> These commands access the CoCoSDC FAT32 filesystem directly from BASIC programs. File numbers and CoCoSDC slots are 0 or 1.

```
SDC_CHAIN "FILENAME.BIN",#
```

Loads a DECB-format machine-language program directly from the CoCoSDC FAT filesystem, replaces the current program, and executes its LOADM postamble address. Include the extension. SDC_CHAIN does not return.

The target must be the normal `.BIN` produced by assembling the compiler's output. Do not process a program that will be loaded by `SDC_CHAIN` with `cc1sl`; `SDC_CHAIN` already contains the loader needed to load a large program across the 64K address space. A compiler-generated first program containing `SDC_CHAIN` should be processed with `cc1sl` when Disk BASIC loads it, even if its main code is small, because the resident loader occupies part of Disk BASIC's loading workspace. See Preparing programs for CHAIN and SDC_CHAIN for the complete build procedure.

EXAMPLE

```
SDC_CHAIN "PART2.BIN",0
```

```
SDC_LOADM "FILENAME.BIN",#[,Offset]
```

Loads a machine-language binary from the CoCoSDC. Offset is optionally added to the original LOADM addresses.

EXAMPLE

```
SDC_LOADM "GAME.BIN",0,256
```

```
x = _SDC_FILEEXISTS(string,#)
```

Returns 255 (-1) when the file exists in the CoCoSDC FAT filesystem, or 0 when it does not.

EXAMPLE

```
A = _SDC_FILEEXISTS(N$,0)
```

SDC_SAVEM "FILENAME.BIN",#,Start,End,Exec

Saves a memory range directly to the CoCoSDC as a machine-language file. Start and End define the inclusive range; Exec is the run address.

EXAMPLE

```
SDC_SAVEM "LEVEL1.BIN",1,28000,29000,28000
```

SDC_BIGLOADM "FILENAME.BIN",#

Loads CoCo 3 memory blocks quickly from a specially prepared BIGLOADM file. This is useful for large assets such as screen backgrounds.

EXAMPLE

```
SDC_BIGLOADM "TITLESCR.BIN",0
```

SDC_OPEN "FILENAME.EXT","X",#

Opens a CoCoSDC file for direct access. Use R for reading or W for writing.

EXAMPLE

```
SDC_OPEN "DATA.DAT","R",0
```

SDC_CLOSE(#)

Closes open CoCoSDC file 0 or 1.

EXAMPLE

```
SDC_CLOSE(0)
```

SDC_PUTBYTE0 x / SDC_PUTBYTE1 x

Writes one byte to open output file 0 or 1. The corresponding file position advances automatically.

EXAMPLE

```
SDC_PUTBYTE0 A
```

x = SDC_GETBYTE(#)

Reads one byte from file 0 or 1. The file position advances automatically.

EXAMPLE

```
B = SDC_GETBYTE(0)
```

x = SDC_LOF(#)

Returns the open file's length as an unsigned 32-bit value (_UNSIGNED LONG) without changing its current position.

EXAMPLE

```
FileSize = SDC_LOF(0)
```

SDC_SETPOS0(x) / SDC_SETPOS1(x)

Sets the corresponding open file's zero-based position to unsigned 32-bit byte offset x. On a file opened for reading, this changes only the read position and writes nothing.

EXAMPLE

```
SDC_SETPOS0(299)
```

A$ = SDC_FILEINFO$(#)

Returns a 32-byte information record. Bytes 1-8 are the name, 9-11 the extension, byte 12 the attributes, and 29-32 the size in least-significant-byte-first order.

EXAMPLE

```
A$ = SDC_FILEINFO$(0)
```

x = SDC_DELETE(A$)

Deletes a file or empty directory at full path A$. Returns 0 for success, 1 if busy too long, 3 for an invalid path, 4 for a hardware error, 5 when not found, or 6 when a directory is not empty.

EXAMPLE

```
R = SDC_DELETE("/GAMES/OLD.BIN")
```

x = SDC_MKDIR(A$)

Creates the directory named by full path A$. Returns 0 for success, 1 if busy too long, 3 for an invalid path, 4 for a hardware error, 5 when the parent is not found, or 6 when the name is already in use.

EXAMPLE

```
R = SDC_MKDIR("/GAMES/NEW")
```

x = SDC_SETDIR(A$)

Changes the current CoCoSDC directory to full path A$. Returns 0 for success, 1 if busy too long, 3 for an invalid path, 4 for a hardware error, or 5 when the directory is not found.

EXAMPLE

```
R = SDC_SETDIR("/GAMES")
```

A$ = SDC_GETCURDIR$(#)

Returns current-directory information for slot 0 or 1. Bytes 1-8 are the filename, 9-11 the extension, and 12-31 are private or reserved.

EXAMPLE

```
A$ = SDC_GETCURDIR$(0)
```

x = SDC_INITDIR(A$)

Initializes a directory listing using a path and wildcard filter. Call it before SDC_DIRPAGE and SDC_DIRLIST$. Returns 0 for success, 1 if busy too long, 3 for an invalid path, 4 for a hardware error, or 5 when the directory is not found.

EXAMPLE

```
R = SDC_INITDIR("MYDIR/*.TXT")
```

x = SDC_DIRPAGE(0)

Retrieves the next 256-byte directory page: sixteen 16-byte records. Continue until a page contains unused zero records. Returns 0 for success, 1 if busy too long, or 4 when not initialized or at end of listing.

EXAMPLE

```
R = SDC_DIRPAGE(0)
```

A$ = SDC_DIRLIST$(y)

Returns entry y (0 through 15) from the latest page. Bytes 1-8 are the name, 9-11 the extension, byte 12 the attributes, and 13-16 the size in most-significant-byte-first order.

EXAMPLE

```
A$ = SDC_DIRLIST$(3)
```

CoCo SDC Movie Playback Commands

> The movie command allows you to playback movies on the CoCo.

```
SDC_PLAYMOVIE "filename",#
```

Will go into graphics mode based on the movie type # is the file number (0 or 1).

EXAMPLE

```
SDC_PLAYMOVIE "MYMOVIE.NTM",0
When the movie is being played you can use number keys to jump to different percentages
of the movie, like 3 will jump to 30% and 9 is 90%, arrow keys jump 60 seconds back or
120 seconds forward and spacebar pauses/resumes and BREAK exits playback.
Creating movies for playback on the CoCo is done with the MakeNTM command line tool.
Your system must also have the full version of FFMPEG and FFPROBE installed.
-gxxx is the same GMODE screen resolution number.
Samples of command line:
./MakeNTM -i=/STS01E01.mkv  -name="Star Trek - The Man Trap" -o=STS01E01.NTM -g136 -
s=.75 -f9 -a2 -k -p1 -c0
./MakeNTM -i=/STS01E01.mkv  -name="Star Trek - The Man Trap" -o=STS01E01.NTM -g136 -
s=.75 -f9 -a2 -k -p1 -c0 -start=0:01:00 -end=0:01:40
./MakeNTM -i=/STS01E28.mkv  -name="The City on the Edge of Forever" -o=STTest.NTM -
length=0:01:00 -g136 -f9 -a2 -s=0.75 -p1 -c0 -k
See the MakeNTM_User_Guide.txt for more help.
```

CoCo SDC Audio Playback Commands

> These commands allow you to playback audio samples/music files directly off the CoCo SDC at a very high bitrate. For mono at 44.75 kHz and stereo at 22.375 kHz

SDC_PLAY "filename",#

Plays an audio file stored on the SDC, outputting sound directly through the CoCo. # is the file number (0 or 1).

EXAMPLE

```
SDC_PLAY "MYAUDIO.RAW",0
```

SDC_PLAYORCL "filename",#

Plays an audio file stored on the SDC, outputting sound through the left channel of the Orchestra 90/CoCo Flash # is the file number (0 or 1).

EXAMPLE

```
SDC_PLAYORCL "MYAUDIO.RAW",0
```

SDC_PLAYORCR "filename",#

Plays an audio file stored on the SDC, outputting sound directly through the CoCo. # is the file number (0 or 1).

EXAMPLE

```
SDC_PLAYORCR "MYAUDIO.RAW",0
```

SDC_PLAYORCS "filename",#

Plays an audio file stored on the SDC, outputting sound directly through the CoCo. # is the file number (0 or 1).

EXAMPLE

```
SDC_PLAYORCS "MYAUDIO.RAW",0
To stop a playing music file or sample you can press the BREAK key.
In order for you to get your audio sample in the correct format to be played back you'll
need to prepare your audio samples and put them on the SD card.  The format for the raw
audio file that will be played is mono 8 bits unsigned.  To convert any sound file or
even the audio from a video file to the correct format used with the SDCPLAY command use
FFMPEG and the following command:
ffmpeg -i source_audio.mp3 -acodec pcm_u8 -f u8 -ac 1 -ar 44750 -af
aresample=44750:filter_size=256:cutoff=1.0 MYAUDIO.RAW
You can also use an audio tool like Audacity to Export an audio sample to an
uncompressed Mono 44750 Hz, RAW (header-less) unsigned 8-bit PCM sample.
If you want to stream 8 bit stereo sound from your CoCo to the COCOFLASH/Orchestra90 use
the command:
SDC_PLAYORCS where the sample MYSAMPLE.RAW is stored on the SD card in your SDC
Controller. It can be created with the FFMPEG command below:
ffmpeg -i source_audio.mp3 -acodec pcm_u8 -f u8 -ac 2 -ar 22375 -af
aresample=22375:filter_size=256:cutoff=1.0 MYSAMPLE.RAW
You can also use an audio tool like Audacity to Export an audio sample to an
uncompressed Stereo 22375 Hz, RAW (header-less) unsigned 8-bit PCM sample.
```

Audio Sample Handling (CoCo 3 only) - Overview

Use the SAMPLE_LOAD SAMPLE_LOAD fileName$,Sample #

and SAMPLE PLAY #, SAMPLE LOOP # OR SAMPLE OFF command

```
' Handle SAMPLE command
' SAMPLE SINGLE 2 ' Play audio sample #2 one single time
' SAMPLE LOOP 4 ' Play audio sample #4 continuously looping
' SAMPLE OFF ' turns off any sample playing
```

ffmpeg -i source_audio.mp3 -acodec pcm_u8 -f u8 -ac 1 -ar 6003 -af aresample=6003:filter_size=256:cutoff=1.0 MYSAMPLE.RAW

Sprite Handling - Overview

To use sprites in your BASIC programs you need to first prepare them with some external tools. - A graphic editor like GIMP to make or convert/export your sprite image into a 32 bit RGBA (includes transparency) PNG file. - The command line tool (included with this compiler) PNGtoCCSB to convert the PNG file into a Sprite to be used in your BASIC program.

```
Typically you'll use PNGtoCCSB to convert your PNG file into a sprite with a command
similar to:
PNGtoCCSB -g15 -s0 MySprite.PNG
PNGtoCCSB -g15 -s0 MyOtherSprite.PNG
```

This command will convert the PNG files MySprite.PNG & MyOtherSprite.PNG into compiled sprites ready to be used -g15 tells the tool we are going to use this sprite with a GMODE 15 graphics screen in your BASIC program. When converting the PNG to a compiled sprite it will match the colours of the PNG to the available colours in the GMODE & Screen mode (-sx option) on the CoCo.

This will save the compiled sprites as MySprite.asm & MyOtherSprite.asm

In your BASIC program you will need to start your program with the following commands:

Sprite_Load "MySprite.asm", 0 ‘ Load sprite as #0 Sprite_Load "MyOtherSprite.asm", 1 ‘ Load sprite as #1

```
GMODE 15,1 'This will reserve graphics page 0 and graphics page 1
GMODE 15,0 'Set the current graphics page to use as 0
GCLS 0 'Colour the graphics screen colour 0
SCREEN 1,0 'Show the graphics screen (you could do this later)
```

Draw any background graphics for your program here...

Once your background graphics are drawn you need to copy screen 0 to screen 1 this is done with the GCOPY command. This is necessary as the sprite routines use double buffering where the screen is switched to show screen 1 the sprite updates are drawn on screen 0, once the sprites are updated, screen 0 is shown and screen 1 sprite updates are drawn.

GCOPY 0,1 ‘Copy graphics screen 0 to graphics screen 1

Now that your background is setup you are ready to draw the sprites on the screen using the following commands:

A normal Sprite usage routine would look like this:

```
SPRITE LOCATE 0, X1, Y1 ' Set the location of sprite 0
SPRITE BACKUP 0     ' Copy what's behind sprite 0
SPRITE SHOW 0, 0        ' Draw sprite 0, frame 0
SPRITE LOCATE 1, X2, Y2 ' Set the location of sprite 1
SPRITE BACKUP 1     ' Copy what's behind sprite 1
SPRITE SHOW 1, 0        ' Draw sprite 1, frame 0
```

Except for the SPRITE LOCATE command the above commands aren't executed at this point, they are added to a sprite command queue that is executed when the WAIT VBL command is used as:

```
WAIT VBL                ' This is when the actual sprite updates occur
```

Typically you will next want the sprite erase commands after the Wait VBL command, which again are only added to the sprite command queue at this point. It is usually best to reverse the order of the sprites for erasing.

```
SPRITE ERASE 1      ' Erase Sprite 1 (reverse order is good)
SPRITE ERASE 0      ' Erase Sprite 0
```

Typically you would have a routine to update your sprites like this:

```
SpriteUpdate:
SPRITE LOCATE 0, X1, Y1 ' Set the location of sprite 0
SPRITE BACKUP 0     ' Copy what's behind sprite 0
SPRITE SHOW 0, 0        ' Draw sprite 0, frame 0
SPRITE LOCATE 1, X2, Y2 ' Set the location of sprite 1
SPRITE BACKUP 1     ' Copy what's behind sprite 1
SPRITE SHOW 1, 0        ' Draw sprite 1, frame 0
WAIT VBL                ' This is when the actual sprite updates occur
SPRITE ERASE 1      ' Erase Sprite 1 (reverse order is good)
SPRITE ERASE 0      ' Erase Sprite 0
Return              ' All done updating sprites
```

In your program you would change X1,Y1 & X2,Y2 to the locations you want and then update the sprites on the screen with the command:

```
GOSUB SpriteUpdate
```

The last thing to know about the sprites is they can also have a frame option. If you create or download a sprite sheet of an animated sprite, they are usually created as many images in a row. For example a sprite sheet might look something like this:

Illustration from the original BasTo6809 manual.

```
If you wanted to use this sprite in your program you would use the -axx option with the
PNGtoCCSB tool in this case we have 8 frames of animation so we would use the following
command:
PNGtoCCSB -g15 -s0 -a8 WolfCub.PNG
```

The -a8 tells the tool to create a compiled sprite with 8 frames from the one PNG file. The PNG must only have the images in one horizontal row and be evenly spaced.

In your program you use the same SPRITE commands as above except you also now want to use the frame option of the SPRITE SHOW command where the 8 frames to show are from frame 0 to frame 7

Sprite_Load "WolfCub.asm", 2 ‘ Load sprite as #2

The sprite update code would now look like this:

```
SpriteUpdate:
SPRITE LOCATE 0, X1, Y1 ' Set the location of sprite 0
SPRITE BACKUP 0     ' Copy what's behind sprite 0
SPRITE SHOW 0, 0        ' Draw sprite 0, frame 0
SPRITE LOCATE 1, X2, Y2 ' Set the location of sprite 1
SPRITE BACKUP 1     ' Copy what's behind sprite 1
SPRITE SHOW 1, 0        ' Draw sprite 1, frame 0
SPRITE LOCATE 2, WolfCubX, WolfCubY ' Set the location of sprite 2
SPRITE BACKUP 2     ' Copy what's behind sprite 2
SPRITE SHOW 2, F        ' Draw sprite 2, frame F
WAIT VBL                ' This is when the actual sprite updates occur
SPRITE ERASE 2      ' Erase Sprite 2 (reverse order is good)
SPRITE ERASE 1      ' Erase Sprite 1
SPRITE ERASE 0      ' Erase Sprite 0
Return              ' All done updating sprites
```

- At this point PNGtoCCSB doesn't support semigraphics screen modes for sprites, but all other screen modes are supported.

CoCo 1 & 2 Graphic Modes - GMODE

GMODE #	Resolution	Colours	Bytes Per Screen	Mode Name
0	32 x 16	9	512	Internal alphanumeric
1	32 x 16	2	512	External alphanumeric
2	64 x 32	9	512	Semi graphic-4
3	64 x 32	9	2048	Semi graphic-8*
4	64 x 48	9**	512	Semi graphic-6
5	64 x 48	9	3072	Semi graphic-12***
6	64 x 64	9	2048	Semi graphic-8
7	64 x 96	9	3072	Semi graphic-12
8	64 x 192	9	6144	Semi graphic-24
9	64 x 64	4	1024	Full graphic 1-C
10	128 x 64	2	1024	Full graphic 1-R
11	128 x 64	4	2048	Full graphic 2-C
12	128 x 96	2	1536	Full graphic 2-R
13	128 x 96	4	3072	Full graphic 3-C
14	128 x 192	2	3072	Full graphic 3-R
15	128 x 192	4	6144	Full graphic 6-C
16	256 x 192	2	6144	Full graphic 6-R

GMODE #	Resolution	Colours	Bytes Per Screen	Mode Name
17			6144	DMA Mode (unusable)
18	256 x 192	2	6144	Compile for Artifacts

- Semi graphics 4 Hybrid using Semi Graphics 8 Mode ** 3 Colours for each colour set *** Semi graphics-6 Hybrid using Semi Graphics 12 mode

CoCo 3 Graphic Modes - GMODE

GMODE #	Resolution	Colours	Bytes Per Screen
100	64 x 192	4	3200
101	64 x 200	4	3200
102	64 x 225	4	3600
103	64 x 192	16	6144
104	64 x 200	16	6400
105	64 x 225	16	7200
106	80 x 192	4	3840
107	80 x 200	4	4000
108	80 x 225	4	4500
109	80 x 192	16	7680
110	80 x 200	16	8000
111	80 x 225	16	9000
112	128 x 192	2	3072
113	128 x 200	2	3200
114	128 x 225	2	3600
115	128 x 192	4	6144
116	128 x 200	4	6400

GMODE #	Resolution	Colours	Bytes Per Screen
117	128 x 225	4	7200
118	128 x 192	16	12288
119	128 x 200	16	12800
120	128 x 225	16	14400
121	160 x 192*	2	3840
*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen
122	160 x 200*	2	4000
*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen
123	160 x 225*	2	4500
*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen	*(viewable) really 128x192, * Special mode that repeats the left 4 bytes on the right side of the screen
124	160 x 192	4	7680

GMODE #	Resolution	Colours	Bytes Per Screen
125	160 x 200	4	8000
126	160 x 225	4	9000
127	160 x 192	16	15360
128	160 x 200	16	16000
129	160 x 225	16	18000
130	256 x 192	2	6144
131	256 x 200	2	6400
132	256 x 225	2	7200
133	256 x 192	4	12288
134	256 x 200	4	12800
135	256 x 225	4	14400
136	256 x 192	16	24576
137	256 x 200	16	25600
138	256 x 225	16	28800
139	320 x 192	2	7680
140	320 x 200	2	8000
141	320 x 225	2	9000
142	320 x 192	4	15360

GMODE #	Resolution	Colours	Bytes Per Screen
143	320 x 200	4	16000
144	320 x 225	4	18000
145	320 x 192	16	30720
146	320 x 200	16	32000
147	320 x 225	16	36000
148	512 x 192	2	12288
149	512 x 200	2	12800
150	512 x 225	2	14400
151	512 x 192	4	24576
152	512 x 200	4	25600
153	512 x 225	4	28800
154	640 x 192	2	15360
155	640 x 200	2	16000
156	640 x 225	2	18000
157	640 x 192	4	30720
158	640 x 200	4	32000
159	640 x 225	4	36000
160*	128 x 192	256	24576

GMODE #	Resolution	Colours	Bytes Per Screen
161*	128 x 200	256	25600
162*	128 x 225	256	28800
163*	160 x 192	256	30720
164*	160 x 200	256	32000
165*	160 x 225	256	36000

- These modes are NTSC composite CoCo 3 output only

Specifications

SDC_BIGLOADM"FILENAME.BIN",# Loads CoCo 3 Memory Blocks very fast. Useful for loading in background screens for games or other large amounts of data. # is the file number 0 or 1

This command loads complete $2000 byte blocks into the CoCo 3's memory. The format of this file is written in 512 byte chunks, as the SDC streaming mode reads file data 512 bytes at a time. Chunk 0 is the header/Memory Mapping Chunk:

Bytes

0 & 1 File version #

Version 1 is the only supported version to date

2 & 3 Is the 16 bit number of the first bank to load

x & x+1 Is the 16 bit number of the next bank to load and so on until we get a $FFFF

```
If we reach a $FFFF this signifies we've copied the last block
```

and are done.

512 & 513 Chunk 1 - This is the actual first word of data for the first block

This contains $2000 bytes of data, the next $2000 bytes will be

the data for the 2nd bank number given in the memory mapping Chunk and will continue with $2000 bytes of data for

all the rest of the banks given in the memory map Chunk.

Example Programs

This is a tweaked version of James Diffendaffer's 3D plot program that I converted from working on a CoCo 3 to work on a CoCo 1 & 2

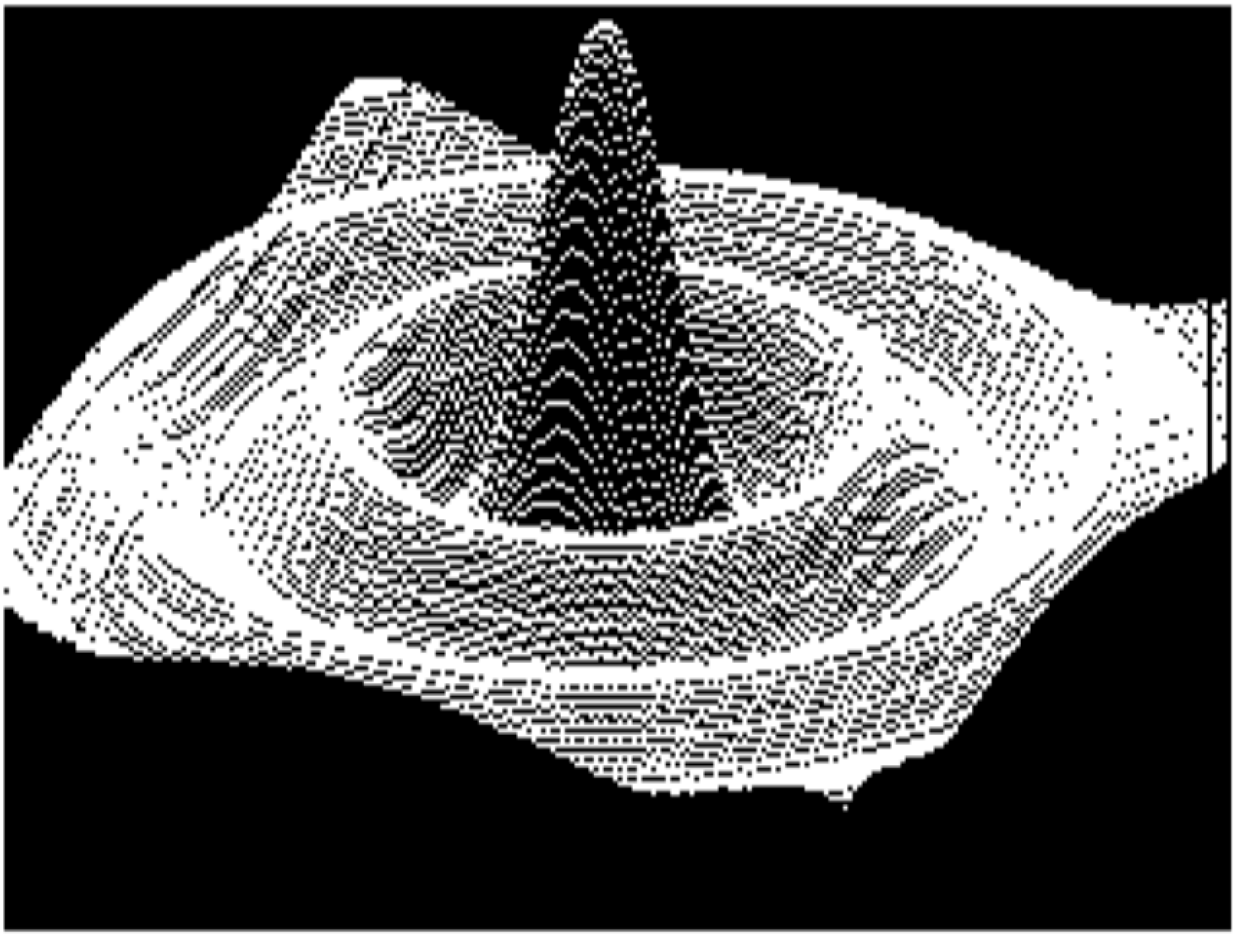

Illustration from the original BasTo6809 manual.

Original program (EXTENDED COLOR BASIC):

```
0 CX=255:CY=192:PMODE 4,1:PCLS:SCREEN 1,1
1 DIM R(255):FOR I=0 TO CX:R(I)=CY:NEXT I:GOTO 10
2 R=SQR(X*X+Y*Y)*1.5: IF R=0 THEN F=90:GOTO 4
3 F=90*SIN(R)/R
4 A=10*X+125-5*Y:B=5*Y+2.5*X+93:RETURN
10 FOR Y=10 TO -10 STEP -0.1
70 FOR X=10 TO -10 STEP -0.1
80 GOSUB 2
82 IF A<0 THEN A=0
83 IF A>255 then A=255
84 IF R(A)>B-F THEN R(A)=B-F:PSET(A,B-F)
90 NEXT X,Y
101 GOTO 101
```

Below is the same program modified to work with the compiler. The only differences are: (The SDECB IDE auto indents the code) 1) The variables are declared with the DIM command 2) GMODE instead of PMODE 3) GCLS instead of PCLS 4) Line 84 the SET command is used which includes a colour value, instead of PSET

```
' compileoptions -s20
' Declare the format Type of numeric variables
DIM CX AS _UNSIGNED _BYTE, CY AS _UNSIGNED _BYTE
DIM R(255) AS SINGLE
DIM A AS SINGLE, B AS SINGLE
DIM F AS SINGLE
DIM R AS SINGLE
DIM X AS SINGLE, Y AS SINGLE
DIM I AS _UNSIGNED _BYTE
0 CX = 255: CY = 192: GMODE 16, 0: GCLS: SCREEN 1, 1
1 FOR I = 0 TO CX: R(I) = CY: NEXT I: GOTO 10
2 R = SQR(X * X + Y * Y) * 1.5: IF R = 0 THEN F = 90: GOTO 4
3 F = 90 * SIN(R) / R
4 A = 10 * X + 125 - 5 * Y: B = 5 * Y + 2.5 * X + 93: RETURN
10 FOR Y = 10 TO -10 STEP -0.1
70 FOR X = 10 TO -10 STEP -0.1
80 GOSUB 2
82 IF A < 0 THEN A = 0
83 IF A > 255 THEN A = 255
84 IF R(A) > B - F THEN R(A) = B - F: SET (A, B - F, 1)
90 NEXT X, Y
101 GOTO 101
```

This is a sample of using a 640x225, 4 colour mode on a CoCo 3 Thanks to bluetip on the CoCo Nation Discord server

Illustration from the original BasTo6809 manual.

```
' compileoptions -s20
' Declare the format of numeric variables
DIM XMAX AS _UNSIGNED INTEGER
DIM X AS _UNSIGNED INTEGER
DIM YMAX AS _UNSIGNED _BYTE
DIM Y AS _UNSIGNED _BYTE
DIM Z AS _UNSIGNED INTEGER
GMODE 159
PALETTE 0, 0
PALETTE 1, 63
PALETTE 2, 18
PALETTE 3, 36
GCLS 0
SCREEN 1, 0
```

XMAX = 639

YMAX = 224

```
FOR Z = 0 TO 32767
SET (RND(XMAX), RND(YMAX), RND(2) + 1)
NEXT Z
FOR Y = 0 TO YMAX STEP 8
LINE (0, Y)-(XMAX, Y), 1
NEXT Y
FOR X = 0 TO XMAX STEP 8
LINE (X, 0)-(X, YMAX), 1
NEXT X
Finish:
GOTO Finish
```

TEST.BAS

```
CoCo 1/2 Hi-res dots
' compileoptions -s20
DIM X AS _UNSIGNED INTEGER
DIM I AS _UNSIGNED _BYTE
10 CLS: PRINT "WELCOME TO THE COCO COMPILER    TEST PROGRAM. IF YOU ARE READINGTHIS THAN
YOU HAVE SUCCESSFULLY COMPILED THE TEST PROGRAM."
20 PRINT: INPUT "PRESS ENTER TO SEE THE FG6R     SCREEN FILL WITH 20,000 DOTS"; A$
30 GMODE 16, 0: GCLS: SCREEN 1, 1
40 FOR X = 1 TO 20000
50 SET (RND(256) - 1, RND(192~%%) - 1, 1)
60 NEXT X
70 SOUND 100, 20
80 SCREEN 0, 0
90 CLS
100 PRINT: PRINT " ALL GOOD, NOW COMPILE YOUR OWN   BASIC PROGRAMS AND HAVE FUN"
110 PRINT: PRINT: PRINT "        COCO FOREVER!!!"
120 FOR I = 1 TO 200 STEP 25
130 SOUND I, 1
140 NEXT I
```

GETFSIZE.BAS

Get the size of a file stored on the actual SD card's filesystem on the CoCoSDC (not the files in a .DSK image)

```
' compileoptions -s20
' Print length of a file on the CoCo SDC
DIM FileLength AS _UNSIGNED LONG ' LONG is a 32 bit number
CLS
PRINT "ENTER THE FILENAME TO GET"
INPUT "THE SIZE OF"; Filename$
SDC_OPEN Filename$, "R", 1 ' Open file for reading
```

A$ = SDC_FILEINFO$(1) ' Get fileinfo in A$

```
SDC_CLOSE (1) 'close file
PRINT: PRINT "RAW HEX CODES OF FILE INFO:"
FOR I = 1 TO LEN(A$)
PRINT RIGHT$("0" + HEX$(ASC(MID$(A$, I, 1))), 2);
NEXT I
PRINT
' Size is stored LSB first
```

FileLength = ASC(MID$(A$, 29, 1)) + ASC(MID$(A$, 30, 1)) * 256 + ASC(MID$(A$, 31, 1)) * 65536 + ASC(MID$(A$, 32, 1)) * 16777216

```
PRINT "LENGTH OF FILE: "; Filename$
PRINT "IS:"; FileLength; " BYTES"
Finished:
GOTO Finished
```

BASASSEM.BAS

Interface BASIC with assembly code

```
' compileoptions -s20
' Example how to interface BASIC and assembly code
DIM N AS INTEGER
5 CLS: PRINT @32,
10 INPUT "ENTER A NUMBER FROM -32000 TO 32000"; N
20 PRINT N; "+ 7 IS";
30 GOSUB 1000 ' Goto assembly code and do an add
35 PRINT N
40 INPUT "WHAT IS YOUR NAME"; Name$
50 GOSUB LearnStrings ' Copy the Name$ string to the top of the screen
60 PRINT "NICE NAME "; Name$
70 END
1000 ' Example of some assembly code
' ADDASSEM:
; Comments in assembly code must start with a semicolon or an asterisks
; Comment is BASIC code must use apostrophe or REM
; Let's access the BASIC numeric variable N
; and multiply it by 2
LDD     _Var_N  ; Get the signed 16 variable N into D
ADDD    #$0007  ; Add 7
STD     _Var_N  ; Store the result back into variable N
; Can have the RTS here or use RETURN just past the ENDASSEM:
' ENDASSEM:
RETURN
LearnStrings:
' ADDASSEM:
; Example of how to access a string variable in assembly code
; Copy the string to the top of the text screen
LDB     _StrVar_Name    ; Get the length of the string
LDU     #_StrVar_Name+1 ; Point at the actual string data
LDX     #$400           ; Point at the top left corner of the text screen
!       LDA ,U+         ; Get the next character
STA     ,X+             ; Store the character on the screen
DECB                    ; Decrement the counter
BNE     <               ; Loop until counter is 0
RTS
' ENDASSEM:
' The compiler adds a prefix to variables (which you can see at the beginning of the
.asm file it genereates)
' numeric variables are prefixed with _Var_ - numeric variables are signed 16 bit
numbers
' string variables are prefixed with _StrVar_ - strings are stored beginning at the
address of the variable
' the format is - byte 0 is the length of the string, the actual string starts at byte 1
'
```

PAGEFLIP.BAS

Example of GMODE, GCOPY, CIRCLE and PAINT commands to do simple animation

```
' compileoptions -s16
' Example of doing page animation
' Must always setup the number of pages you will use as the
' first GMODE command in your program
' With this many pages you will require 64k and will need to use
' the cc1sl (CoCo Super Loader program) see the Manual.pdf
DIM S AS _BYTE ' Range of -128 to 127
DIM X AS _UNSIGNED _BYTE ' Range from 0 to 255
DIM Frames AS _UNSIGNED _BYTE ' RANGE 0 to 255
' First GMODE sets graphic mode and reserves 7 pages 0 to 6
GMODE 16, 6
GMODE , 0 ' Set the active graphics page to 0
GCLS 0 ' Colour the screen colour 0 which is black
```

Frames = 6 ' Number of frames to use

```
FOR X = 1 TO 2000 ' Draw some dots on the screen
SET (RND(256) - 1, RND(192) - 1, 1)
NEXT X
FOR X = 1 TO Frames ' Copy page 0 to all the other pages
40 GCOPY 0, X
NEXT X
' Draw and fill a circle on the screen
50 X = 20
FOR S = 0 TO Frames
GMODE , S
CIRCLE (X, 20), 11, 1
PAINT (X, 20), 0, 1
```

X = X + 10

```
NEXT S
' Ping Pong animation frames (Loops forever)
100 FOR S = 0 TO Frames
GMODE , S: SCREEN 1, 1 ' Screen shows the selected page
SOUND 10, 5
NEXT S
FOR S = Frames - 1 TO 1 STEP -1
GMODE , S: SCREEN 1, 1 ' Screen shows the selected page
SOUND 10, 5
NEXT S
GOTO 100
```

SDCDIR.BAS

Show a directory of the files on the CoCoSDC FAT32 filesystem

```
' compileoptions -s20
DIM X AS _BYTE
DIM Y AS _UNSIGNED _BYTE
CLS: PRINT "SDC ACCESS"
```

CurrentDir$ = "." ' Start at the top folder on the SDC

```
' CurrentDir$="TEST1/*.*" ' Start at the folder TEST on the SDC
PRINT "PATH:"; CurrentDir$
' Set the initial directory location
```

X = SDC_INITDIR(CurrentDir$): GOSUB CheckForError

```
PRINT "DIRECTORY:"
NextPage:
```

X = SDC_DIRPAGE(0): GOSUB CheckForError ' Get 16 directory entries

Y = 0

```
WHILE Y < 16
```

A$ = SDC_DIRLIST$(Y) ' A$ = The 16 byte entry

```
IF ASC(LEFT$(A$, 1)) = 0 THEN END ' If entry is blank then we've reached the last entry
FOR X = 1 TO 8: PRINT MID$(A$, X, 1);: NEXT X
PRINT ".";
FOR X = 9 TO 11: PRINT MID$(A$, X, 1);: NEXT X
PRINT
```

Y = Y + 1

```
WEND
GOTO NextPage
CheckForError:
IF X <> 0 THEN PRINT "ERROR NUMBER IS:"; X: END
RETURN
```

CONSTANT.BAS

Show an example of using constants in your program.

```
' compileoptions -s20
' Declare a numeric constant approximately equal to the ratio of a circle's
' circumference to its diameter:
Const PI = 3.141593
' Declare some string constants:
Const circumferenceText = "THE CIRCUMFERENCE OF THE CIRCLE IS"
Const areaText = "THE AREA OF THE CIRCLE IS"
Do
Input "ENTER THE RADIUS OF A CIRCLE OR ZERO TO QUIT"; radius
If radius = 0 Then End
Print circumferenceText; 2 * PI * radius
Print areaText; PI * radius * radius ' radius squared
Print
Loop
End
```

Thanks

I'd like to thank Scott Cooper (Tazman) for initial testing of the compiler. Scott also wrote the assembly code for the SET and POINT routines used in the semi-graphics modes. Thanks to Curtis Boyle for teaching me how to do horizontal scrolling on the CoCo 3.

I'd also like to thank others on The CoCo Nation Discord who inspired me to keep adding new features, including Bruce D. Moore, Erico Monteiro & Pete Willard.

Thanks also goes to William Astle for writing the awesome 6809 assembler lwasm and for allowing me to include the binary versions with my compiler.

Thanks also goes to Boisy Pitre for allowing me to include his decb (CoCo DISK image) tool with the binary version of my compiler.

Thanks to bluetip